

# Manual de entrega - API REST NestJS, MySQL y RabbitMQ

Laboratorio: Optativa IV - Sistemas de Tiempo Real Distribuidos

Fundación Escuela Tecnológica de Neiva (FET)

Taller 2: Mensajería con RabbitMQ en contenedores Docker

## 1. Contenido del paquete de entrega

El ZIP contiene el código fuente (sin node\_modules), Docker, el script SQL, las colecciones de Postman, las capturas y este manual. El PDF está en la raíz de la carpeta de entrega y también en API\_NEST/docs/.

- Código fuente: carpeta API\_NEST/
- Imagen y composición: Dockerfile y docker-compose.yml
- Script SQL: sql/01\_create\_database\_and\_clients.sql
- Postman de clientes: postman/API\_Nest\_Clients.postman\_collection.json
- Postman de mensajería: postman/Mensajeria\_RabbitMQ.postman\_collection.json
- Capturas: postman/\*.png y postman/\*.jpeg
- Este manual: Manual\_Entrega\_API\_Nest.pdf

## 2. Repositorio

El código también está publicado en:

[https://github.com/SamirCortes/API\\_NEST.git](https://github.com/SamirCortes/API_NEST.git)

```
git clone https://github.com/SamirCortes/API_NEST.git
cd API_NEST
```

## 3. Cómo levantar todo con un solo comando

En el equipo solo hace falta Docker. No se instala Node, RabbitMQ ni MySQL. Con Docker encendido:

```
cd API_NEST
docker compose up -d --build
```

Ese comando construye la imagen y levanta MySQL, RabbitMQ, la API productora y el consumidor. La cola messages\_queue es durable y se crea sola al conectar.

- API productora: http://localhost:3020
- Swagger: http://localhost:3020/api/docs
- MySQL en el equipo: localhost:3307 (usuario nest, clave nest)
- RabbitMQ AMQP: localhost:5672
- Consola RabbitMQ: http://localhost:15672 (usuario nest, clave nest)

Dentro de la red de Docker los servicios se llaman mysql y rabbitmq, no localhost. La API publica el puerto 3020 porque en este equipo el 3000 estaba ocupado. Dentro del contenedor el proceso escucha en el 3000.

```
docker compose logs -f consumer
docker compose logs -f api
docker compose down
```

## 4. Productor

El contenedor de la API expone POST /messages. Recibe un JSON, lo publica en messages\_queue y responde Content-Type: application/json. El campo status es booleano.

```
{"sensor": "OD-01", "valor": 4.2, "unidad": "mg/L"}
```

**Si el mensaje se publica:**

```
{"status": true, "message": "Mensaje encolado"}
```

**Si el cuerpo no es JSON válido:**

```
{"status": false, "message": "Formato de mensaje inválido"}
```

Crear, actualizar o borrar un cliente también publica un evento en la misma cola. Consultar o listar no publica nada.

## 5. Consumidor

El contenedor `api_nest_consumer` usa la misma imagen y ejecuta `node dist/consumer.js`. No abre un puerto. Se conecta a `messages_queue`, escribe en su log el contenido y la hora, y solo después confirma el mensaje para retirarlo de la cola. Si RabbitMQ aún no está listo, reintenta la conexión.

```
docker compose logs -f consumer
```

## 6. Casos que se comprueban

### 1. Consumidor detenido

```
docker compose stop consumer
# Publicar varios POST /messages
# En la consola, messages_queue muestra Ready > 0
docker compose start consumer
docker compose logs -f consumer
```

Los mensajes se acumulan y, al levantar el consumidor, se procesan todos. La cola vuelve a Ready 0.

### 2. Reinicio de RabbitMQ

Con el consumidor detenido y mensajes en Ready se ejecuta `docker compose restart rabbitmq`. La cola es durable y los mensajes son persistentes, así que siguen en la cola al volver la consola. En la captura de los 5 mensajes se ve `durable: true` y `Persistent: 5`.

```
docker compose restart rabbitmq
```

### 3. Mensaje inválido

```
curl -i -X POST http://localhost:3020/messages \
-H "Content-Type: application/json" \
-d '{sensor: OD-01}'
```

La respuesta lleva status en false y el contador de la cola no aumenta.

## 7. Colecciones de Postman

### Mensajería

```
postman/Mensajería_RabbitMQ.postman_collection.json
```

Importar en Postman. La variable `baseUrl` es `http://localhost:3020`. La petición 1 envía un mensaje válido y la 2 un cuerpo que no es JSON. Para diez mensajes válidos, usar el Collection Runner solo con la petición 1 y 10 iteraciones.

### Cientes

```
postman/API_Nest_Clients.postman_collection.json
```

Ejecutar las peticiones del CRUD en orden. `baseUrl` también apunta al puerto 3020.

## 8. API de clientes

- POST /clients crea un cliente y publica `client.created`
- GET /clients y GET /clients/:id solo consultan
- PUT /clients/:id actualiza y publica `client.updated`
- DELETE /clients/:id elimina y publica `client.deleted`

La tabla clients se crea con la migración al arrancar la API, o con sql/01\_create\_database\_and\_clients.sql.

## 9. Declaración de uso de herramientas de inteligencia artificial

Se utilizó Cursor, con el modelo Grok, como asistencia durante el laboratorio.

### Para qué se usó:

- Comparar el taller de RabbitMQ con la API de clientes ya existente
- Separar el productor y el consumidor en contenedores distintos
- Publicar mensajes persistentes en la cola durable messages\_queue
- Ajustar POST /messages al formato de respuesta pedido por el taller
- Actualizar Docker Compose, el README, Swagger y la colección de Postman
- Generar este manual y el paquete ZIP de entrega

### Qué se revisó o hizo sobre ese resultado:

- Pruebas manuales en Swagger y Postman, incluidas las capturas
- Comprobación de los tres casos: consumidor detenido, reinicio y mensaje inválido
- El puerto publicado de la API quedó en 3020 porque el 3000 estaba ocupado

El estudiante asume la responsabilidad del código entregado, de su funcionamiento y de la comprensión de la solución.

## 10. Evidencias del CRUD de clientes

Capturas de Postman de las peticiones del laboratorio anterior.

## 1. Crear cliente (POST /clients)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API endpoints under "API Nest - Clients". The main panel shows a POST request to "1. Crear cliente" with a body of JSON data. The response is a 201 status code with a detailed JSON object.

**Request:**

```
POST {{baseUrl}}/clients
```

**Body (JSON):**

```
{  "names": "Jose",  "surnames": "pacheco",  "age": 30,  "status": true}
```

**Response (201 Created):**

```
{  "status": true,  "message": "Registro creado",  "data": {    "id": 4,    "names": "Jose",    "surnames": "pacheco",    "age": 30,    "createdAt": "2026-09-21T02:37:04.185Z",    "status": true  }}
```

The bottom status bar indicates "Import Complete".

## 2. Listar clientes (GET /clients)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API endpoints under "API Nest - Clients". The main panel shows a GET request to "GET 2. Listar clientes" with the URL "GET {{baseUri}} /clients". The response is a 200 OK status with a response time of 16 ms and a body size of 718 B. The response body is a JSON array of two client objects.

**Headers**

| Key | Value | Description |
|-----|-------|-------------|
| Key | Value | Description |

**Body**

```
{
  "data": [
    {
      "id": 1,
      "names": "Ana",
      "suinames": "Lopez",
      "age": 28,
      "createdAt": "2026-09-21T02:25:49.897Z",
      "status": true
    },
    {
      "id": 2,
      "names": "Jorge",
      "suinames": "Olaya",
      "age": 28,
      "createdAt": "2026-09-21T02:26:25.664Z",
      "status": true
    }
  ]
}
```

### 3. Consultar cliente por id (GET /clients/:id)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API endpoints under "API Nest - Clients". The selected endpoint is "GET 3. Consultar cliente por id". The main panel shows the request details for this endpoint, including the method "GET", the URL "http://localhost:3000/clients/4", and the response body.

**Request Details:**

- Method: GET
- URL: `http://localhost:3000/clients/{{clientId}}`
- Query Params: A table with 2 columns: Key, Value, Description, Bulk Edit. The first row has "Key" in the Key column and "Value" in the Value column.

**Response Details:**

- Status: 200 OK
- Time: 15 ms
- Size: 397 B
- Response Body (JSON):

```
1 {
2   "status": true,
3   "message": "Registro consultado",
4   "data": {
5     "id": 4,
6     "names": "Jose",
7     "surnames": "pacheco",
8     "age": 30,
9     "createdAt": "2026-09-21T02:37:04.185Z",
10    "status": true
11  }
12 }
```

## 4. Actualizar cliente (PUT /clients/:id)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API endpoints under "API Nest - Clients". The selected endpoint is "PUT 4. Actualizar cliente". The main panel shows the request configuration for a PUT request to the URL `{{baseUrl}}/clients/{{clientId}}`. The response is displayed in the "Body" tab, showing a JSON object with a status of `true`, a message of "Registro actualizado", and client data including id, names, surnames, age, and createdAt.

**Request Configuration:**

- Method: PUT
- URL: `{{baseUrl}}/clients/{{clientId}}`
- Environment: No environment

**Response (200 OK):**

```
1 {
2   "status": true,
3   "message": "Registro actualizado",
4   "data": {
5     "id": 4,
6     "names": "Ana",
7     "surnames": "Perez",
8     "age": 30,
9     "createdAt": "2026-09-21T02:37:04.185Z",
10    "status": true
11  }
12 }
```

## 5. Eliminar cliente (DELETE /clients/:id)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API endpoints under "API Nest - Clients". The main panel shows a DELETE request to the endpoint `DELETE {{baseUri}}/clients/{{clientId}}`. The response is a 200 OK status with a JSON body indicating successful deletion.

**Request Details:**

- Method: DELETE
- URL: `DELETE {{baseUri}}/clients/{{clientId}}`
- Params: None

**Response Details:**

- Status: 200 OK
- Time: 24 ms
- Size: 281 B
- Body (JSON):

```
{  "status": true,  "message": "Registro eliminado"}
```

## 6. Listado posterior a eliminar (GET /clients)

The screenshot shows the Postman interface with a workspace named "EDWIN SAMIR CORTES OLAYA's Workspace". The left sidebar displays a collection of API requests under "API Nest - Clients". The selected request is "GET 3. Consultar cliente por id". The request details show a GET method with the URL "http://localhost:3000/clients/{{clientId}}". The response is a 404 Not Found status, indicating the client was not found. The response body is JSON, showing the following structure:

```
{  "status": false,  "message": "Registro no encontrado"}
```

The response status is 404 Not Found, with a response time of 35 ms and a size of 293 B. The response body is JSON, showing the following structure:

```
{  "status": false,  "message": "Registro no encontrado"}
```

## 11. Evidencias de RabbitMQ

Capturas del productor y de la consola de administración. La cola es messages\_queue.

Mensaje válido. POST /messages responde status true y Mensaje encolado.

The screenshot shows a web browser window with a REST client interface. The address bar shows the URL `localhost:3020/api/docs#/messages/MessagesController_publish`. The interface includes a text area for the request body, a blue "Execute" button, and a "Clear" button. Below the "Execute" button, the "Responses" section is visible, showing the "Curl" command, the "Request URL", the "Server response" status (200), and the "Response body" (a JSON object with `"status": true` and `"message": "Mensaje encolado"`). The "Response headers" section is also visible, showing various headers like `access-control-allow-methods`, `access-control-allow-origin`, `connection`, `content-length`, `content-type`, `date`, `etag`, `keep-alive`, and `x-powered-by`.

```
{
  "sensor": "00-01",
  "valor": 4.2,
  "unidad": "mg/L"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3020/messages' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "sensor": "00-01",
    "valor": 4.2,
    "unidad": "mg/L"
  }'
```

Request URL

```
http://localhost:3020/messages
```

Server response

Code Details

200

Response body

```
{
  "status": true,
  "message": "Mensaje encolado"
}
```

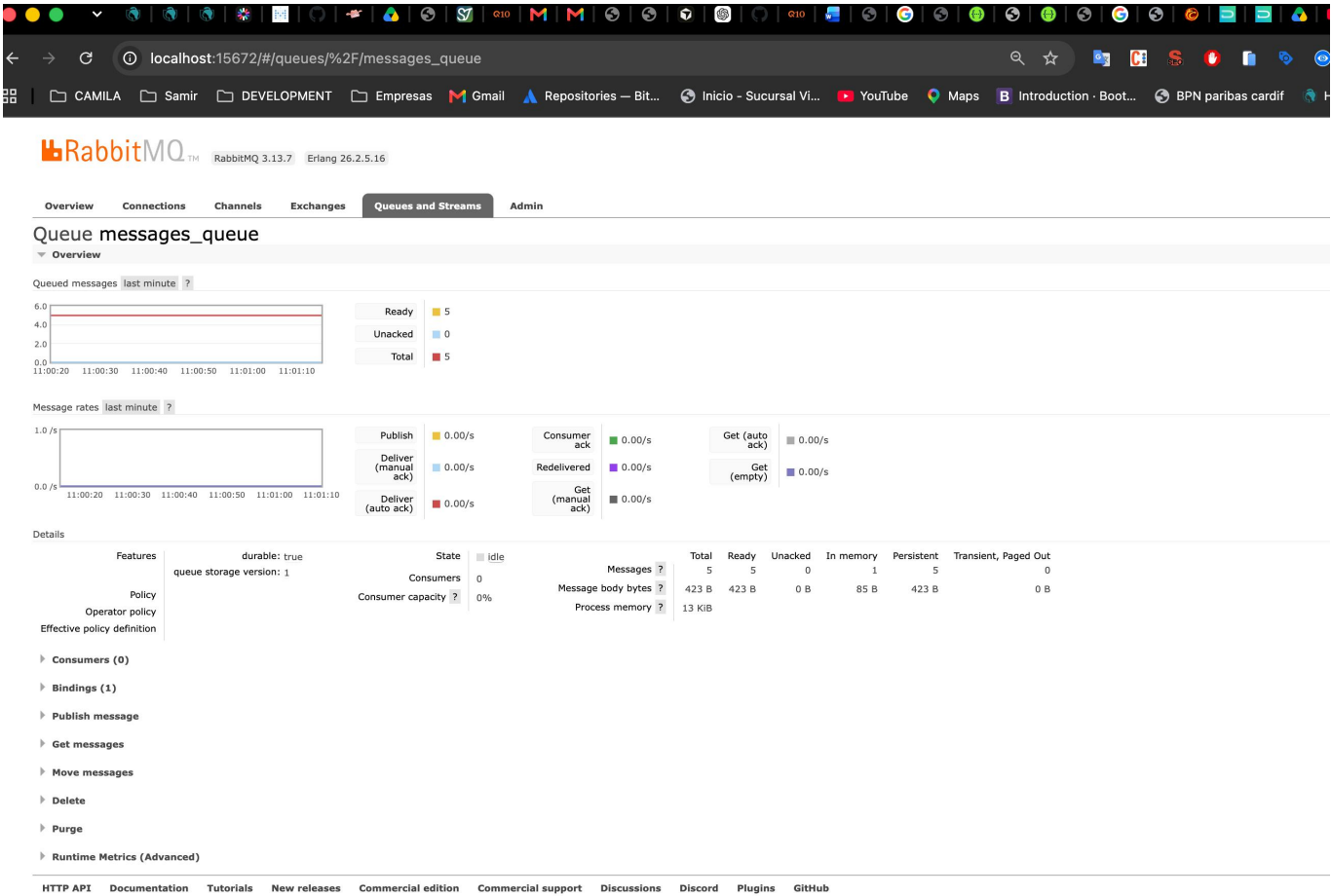
Response headers

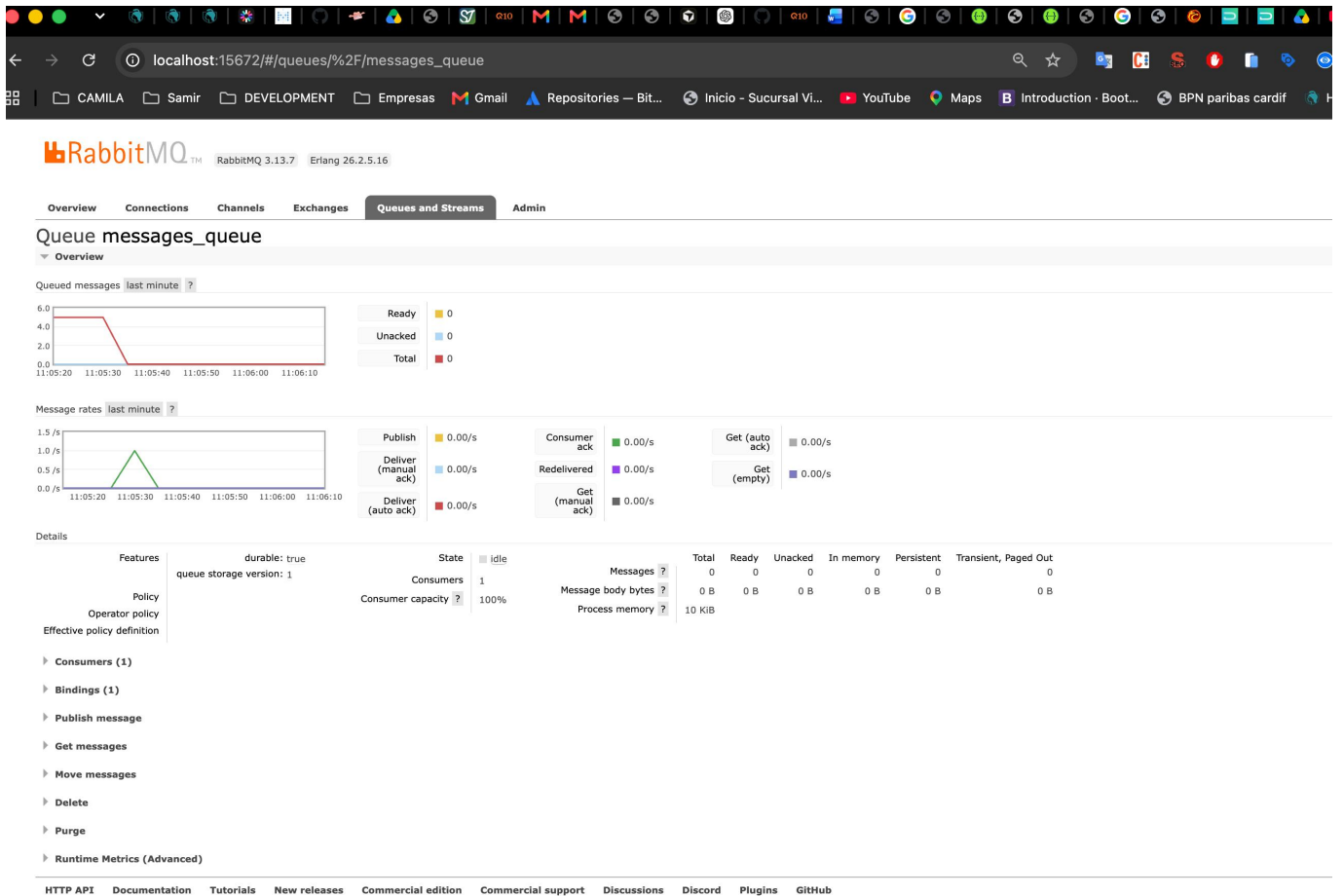
```
access-control-allow-methods: PUT,GET,HEAD,POST,DELETE,OPTIONS
access-control-allow-origin: http://localhost:3020
connection: keep-alive
content-length: 44
content-type: application/json; charset=utf-8
date: Mon, 09 Oct 2023 18:12:00 GMT
etag: W/"2c-HjSJAtbuhwPAChs97Xo1ei4Qz8"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

| Code | Description | Link |
|------|-------------|------|
|------|-------------|------|

Caso 1, antes. Consumidor detenido: 5 mensajes en Ready. La cola es durable y los 5 mensajes son persistentes.



**Caso 1, después. Al levantar el consumidor la cola queda en 0 y aparece 1 consumidor.**

### Caso 3. Cuerpo que no es JSON válido: status false y Formato de mensaje inválido. No se publica en la cola.

The screenshot shows a web browser window with the address bar displaying `localhost:3020/api/docs#/messages/MessagesController_publish`. The browser's taskbar at the top shows various open applications. The main content area displays a REST client interface with the following sections:

- Execute** and **Clear** buttons at the top.
- Responses** section:

  - Curl**:

```
curl -X 'POST' \
  'http://localhost:3020/messages' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{sensor: 00-01}'
```
  - Request URL**: `http://localhost:3020/messages`
  - Server response**:
    - Code**: 400
    - Details**: Error: Bad Request
    - Response body**:

```
{
  "status": false,
  "message": "Formato de mensaje inválido"
}
```
    - Response headers**:

```
access-control-allow-methods: PUT, GET, HEAD, POST, DELETE, OPTIONS
access-control-allow-origin: http://localhost:3020
connection: keep-alive
content-length: 57
content-type: application/json; charset=utf-8
date: Mon, 05 Oct 2026 16:13:30 GMT
etag: W/"29-fm22U+0Upw/f15J7dV88fzquZ5c"
keep-alive: timeout=5
x-powered-by: Express
```

- Responses** table:

| Code | Description      | Links    |
|------|------------------|----------|
| 200  | Mensaje encolado | No links |

Below the table, there is a **Media type** dropdown menu currently set to `application/json`.